开源联盟链平台与智能合约开发培训资源

开源联盟链平台搭建与运维操作

> 以下操作基于FISCO BCOS官方提供的build_chain.sh脚本

vim 进行 ipconf文件编辑（3机构3群组）

```
127.0.0.1:1 agencyA 1,2,3
127.0.0.1:1 agencyB 1,2,3
127.0.0.1:1 agencyC 1,2,3
```

建链：

若有nodes文件夹，先删除nodes文件夹

`bash build_chain.sh -e ./fisco-bcos  -f ipconf`

```
[root@node1 chain]# vim ipconf
[root@node1 chain]# bash build_chain.sh -e ./fisco-bcos  -f ipconf
Checking fisco-bcos binary...
Binary check passed.
================================================================
Generating CA key...
================================================================
Generating keys and certificates ...
Processing IP=127.0.0.1 Total=1 Agency=agencyA Groups=1,2,3
Processing IP=127.0.0.1 Total=1 Agency=agencyB Groups=1,2,3
Processing IP=127.0.0.1 Total=1 Agency=agencyC Groups=1,2,3
================================================================
Generating configuration files ...
Processing IP=127.0.0.1 Total=1 Agency=agencyA Groups=1,2,3
Processing IP=127.0.0.1 Total=1 Agency=agencyB Groups=1,2,3
Processing IP=127.0.0.1 Total=1 Agency=agencyC Groups=1,2,3
================================================================
Group:1 has 3 nodes
Group:2 has 3 nodes
Group:3 has 3 nodes
================================================================
[INFO] FISCO-BCOS Path : ./fisco-bcos
[INFO] IP List File    : ipconf
[INFO] Start Port      : 30300 20200 8545
[INFO] Server IP       : 127.0.0.1:1 127.0.0.1:1 127.0.0.1:1
[INFO] Output Dir      : /root/ucloud/cdgy/chain/nodes
[INFO] CA Path         : /root/ucloud/cdgy/chain/nodes/cert/
[INFO] RSA channel     : true
================================================================
[INFO] Execute the download_console.sh script in directory named by IP to get FISCO-BCOS console.
e.g.  bash /root/ucloud/cdgy/chain/nodes/127.0.0.1/download_console.sh -f
================================================================
[INFO] All completed. Files in /root/ucloud/cdgy/chain/nodes
[root@node1 chain]#
```

`cd nodes/127.0.0.1/` 进入文件夹

`bash start_all.sh` 启动全部节点

`ps -ef |grep fisco-bcos|grep -v grep` 查询节点进程

```
[root@node1 127.0.0.1]# ps -ef |grep fisco-bcos|grep -v grep
root     2309284       1  1 13:23 pts/0    00:00:00 /root/ucloud/cdgy/chain/nodes/127.0.0.1/node0/../fisco-bcos -c config.ini
root     2309290       1  1 13:23 pts/0    00:00:00 /root/ucloud/cdgy/chain/nodes/127.0.0.1/node2/../fisco-bcos -c config.ini
root     2309318       1  1 13:23 pts/0    00:00:00 /root/ucloud/cdgy/chain/nodes/127.0.0.1/node1/../fisco-bcos -c config.ini
```

`tail -f node0/log/* |grep +++` 获取节点共识状态（等待输出内容,ctrl+c结束）

```
[root@node1 127.0.0.1]# tail -f node0/log/* |grep +++
info|2025-04-17 13:24:46.055995|PBFTSeal-1-0x00007f629fe33700|[g:1][CONSENSUS][SEALER]++++++++++++++++ Generating seal on,blkNum=1,tx=0,nodeIdx=2,hash=5663c28d...
info|2025-04-17 13:24:46.162109|PBFTSeal-2-0x00007f629ce2d700|[g:2][CONSENSUS][SEALER]++++++++++++++++ Generating seal on,blkNum=1,tx=0,nodeIdx=2,hash=143c72ba...
info|2025-04-17 13:24:46.245079|PBFTSeal-3-0x00007f6299e27700|[g:3][CONSENSUS][SEALER]++++++++++++++++ Generating seal on,blkNum=1,tx=0,nodeIdx=2,hash=b2a5ec11...
info|2025-04-17 13:24:49.065390|PBFTSeal-1-0x00007f629fe33700|[g:1][CONSENSUS][SEALER]++++++++++++++++ Generating seal on,blkNum=1,tx=0,nodeIdx=2,hash=53cd6ac1...
info|2025-04-17 13:24:49.170591|PBFTSeal-2-0x00007f629ce2d700|[g:2][CONSENSUS][SEALER]++++++++++++++++ Generating seal on,blkNum=1,tx=0,nodeIdx=2,hash=418f26b2...
info|2025-04-17 13:24:49.253460|PBFTSeal-3-0x00007f6299e27700|[g:3][CONSENSUS][SEALER]++++++++++++++++ Generating seal on,blkNum=1,tx=0,nodeIdx=2,hash=3859f131...
```

控制台部署

拷贝节点证书文件到控制台conf目录下

`cp -r nodes/127.0.0.1/sdk/* console/conf/`

```
[root@node1 chain]# cp -r nodes/127.0.0.1/sdk/* console/conf/
cp: overwrite 'console/conf/ca.crt'? y
cp: overwrite 'console/conf/cert.cnf'? y
cp: overwrite 'console/conf/sdk.crt'? y
cp: overwrite 'console/conf/sdk.key'? y
```

`bash start.sh` 启动控制台

```
[root@node1 console]# bash start.sh
==================================================================================
Welcome to FISCO BCOS console(2.10.0)!
Type 'help' or 'h' for help. Type 'quit' or 'q' to quit console.
 _________ _______  _______  _______      _______  _______  _______  _______
|         |      \/      \ /      \ /      \     |       \ /      \ /      \ /      \
| $$$$$$$$\$$$$$$|  $$$$$$|  $$$$$$|  $$$$$$\    | $$$$$$$|  $$$$$$|  $$$$$$|  $$$$$$\
| $$__     | $$ | $$___\$| $$   \$| $$  | $$    | $$__/ $| $$   \$| $$  | $| $$___\$$
| $$  \    | $$  \$$    \| $$     | $$  | $$    | $$    $| $$     | $$  | $$\$$    \
| $$$$$    | $$  _\$$$$$$| $$   __| $$  | $$    | $$$$$$$| $$   __| $$  | $$_\$$$$$$\
| $$      _| $$_|  \__| $| $$__/  | $$__/ $$    | $$__/ $| $$__/  | $$__/ $|  \__| $$
| $$     |   $$ \\$$    $$\$$    $$\$$    $$    | $$    $$\$$    $$\$$    $$\$$    $$
 \$$      \$$$$$$ \$$$$$$  \$$$$$$  \$$$$$$      \$$$$$$$  \$$$$$$  \$$$$$$  \$$$$$$

==================================================================================
[group:1]>
```

测试部署合约是否正常

```
[group:1]> deploy HelloWorld
transaction hash: 0x206dd7f0c99331588056370f677984bc6dfee5bff3fbe5f6016a0f0183471f60
contract address: 0xffe2790036bf378d87bfdd00d9ea68728837827c
currentAccount: 0xe9781bcf192466c8d91062409ada3c11a7bbe06c
```

节点扩容

进入nodes/127.0.0.1/目录

`bash gen_node_cert.sh -c ../cert/agencyA -o node3` 根据实际来指定node后的数字，这里是节点4

```
[root@node1 127.0.0.1]# bash gen_node_cert.sh -c ../cert/agencyA -o node3
================================================================
[INFO] Cert Path    : ../cert/agencyA
[INFO] Output Dir   : node3
[INFO] RSA cert     : true
[INFO] RSA SDK cert     : true
================================================================
[INFO] All completed. Files in node3
```

`cp node0/config.ini node0/*.sh node3/` 拷贝节点配置文件和脚本到新节点

`cp node0/conf/group.* node3/conf/` 拷贝节点的群组文件到新节点

编辑新节点的config.ini文件为如下：

```
[rpc]
    channel_listen_ip=0.0.0.0
    channel_listen_port=20203
    jsonrpc_listen_ip=127.0.0.1
    jsonrpc_listen_port=8548
    disable_dynamic_group=false
[p2p]
    listen_ip=0.0.0.0
    listen_port=30303
    ; nodes to connect
    node.0=127.0.0.1:30300
    node.1=127.0.0.1:30301
    node.2=127.0.0.1:30302
    node.3=127.0.0.1:30303
```

进入节点目录启动节点

```
[root@node1 node3]# bash start.sh
 node3 start successfully
```

在node3目录`cat conf/node.nodeid`获取节点id并拷贝

```
[root@node1 node3]# cat conf/node.nodeid
4555f912f2d19158a0739dc01af1124811f9f89f7db109a352f95f19ee8a108ce980e5a0bff733f6521a30ec26839a080ae86f161dce58bcfa7efc0d218fea26
```

启动控制台并进行如下操作，此时成功将新节点加入到群组中：

```
[group:1]> addSealer 4555f912f2d19158a0739dc01af1124811f9f89f7db109a352f95f19ee8a108ce980e5a0bff733f6521a30ec26839a080ae86f161dce58bcfa7efc0d218fea26
{
    "code":1,
    "msg":"Success"
}

[group:1]> getSealerList
[
    2250a65c80e643b0a2120976e8072e98f792cb7b3ab9214d7e0bef93d18760eb91aa2d1c7098ecd7e808b3ff739628bc236295d6dc966a5dfd8e69bab188719c,
    3a0a3886afdea08a87c50f4e8618469376599df32ef5f4362e727d325eae32fc4ead315822d51fa51fea80fe1c1fb44956058ea4fd6f38959971553ded4a7b62,
    9f0f4e4aeadd14cb8646424fb8c8ce90a8a567bc73bf43e9dac60ba2041575d98a1d60a3d4fb135b1f027e3a3336da6b5bebcff35738be3550566318fed12002,
    4555f912f2d19158a0739dc01af1124811f9f89f7db109a352f95f19ee8a108ce980e5a0bff733f6521a30ec26839a080ae86f161dce58bcfa7efc0d218fea26
]
```

区块链前置服务部署

> 区块链前置服务能够用于编写、部署及调用合约，还能够进行区块链网络的全面监控

`cp -r nodes/127.0.0.1/sdk/* webase-front/conf/`复制节点证书文件到conf下

```
[root@node1 chain]# cp -r nodes/127.0.0.1/sdk/* webase-front/conf/
cp: overwrite 'webase-front/conf/ca.crt'? y
cp: overwrite 'webase-front/conf/cert.cnf'? y
cp: overwrite 'webase-front/conf/sdk.crt'? y
cp: overwrite 'webase-front/conf/sdk.key'? y
```

`bash start.sh`启动

```
[root@node1 webase-front]# bash start.sh
==========================================================================================
Server com.webank.webase.front.Application Port 5002 ...PID(2311885) [Starting]. Please check message through the log file (default path:./log/).
==========================================================================================
```

稍等8秒左右通过`cat log/WeBASE-Front.log |grep "main run success"`查看日志判断是否启动成功

```
[root@node1 webase-front]# cat log/WeBASE-Front.log |grep "main run success"
2025-03-29 13:27:41.741 [main] INFO  Application() - main run success...
2025-04-17 13:42:44.136 [main] INFO  Application() - main run success...
```

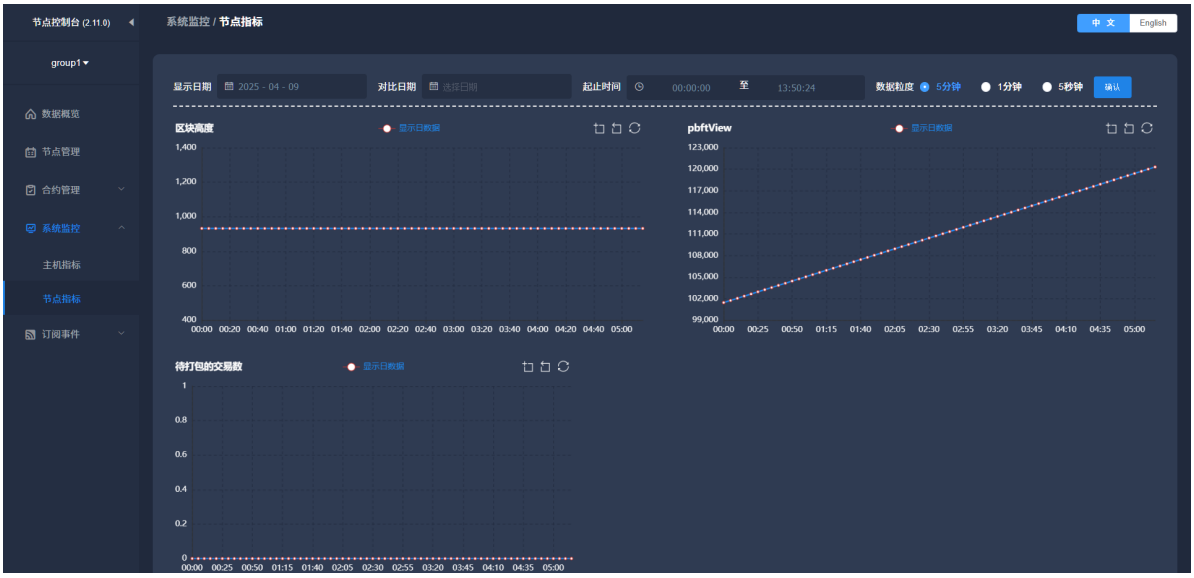
通过`netstat -ntpl |grep 5002`查看端口占用情况

```
[root@node1 webase-front]# netstat -ntpl |grep 5002
tcp6       0      0 :::5002                 :::*                    LISTEN      2311885/java
```

通过浏览器以ip+5002端口来访问webase-front

区块链网络监控（5002端口）

节点指标的监控



区块链主机的监控

智能合约开发

一、Solidity 常识入门

1. 什么是 Solidity?

- Solidity 是一种**静态类型、面向合约的高级编程语言**，专为在以太坊等 EVM 区块链上编写**智能合约**而设计。
- 语法类似 JavaScript，但运行在去中心化的区块链网络中。
- 合约一旦部署，**不可更改**（除非设计了可升级机制）。

2. 智能合约是什么？

- 一段部署在区块链上的代码，自动执行预设逻辑。
- 无需中介，具有**透明、不可篡改、可验证**的特性。

3. 开发工具链

- **Remix IDE**：在线 IDE，适合学习（https://remix.ethereum.org）
- **Hardhat / Foundry**：本地开发框架
- **MetaMask**：钱包插件，用于测试交易
- **Ropsten / Sepolia / Goerli 测试网**：测试用区块链

二、第一个 Solidity 合约

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract HelloWorld {
    string public message = "Hello, World!";

    function setMessage(string memory newMsg) public {
        message = newMsg;
    }

    function getMessage() public view returns (string memory) {
        return message;
    }
}
```

代码解析：

语法	说明
`// SPDX-License-Identifier: MIT`	开源许可证声明
`pragma solidity ^0.8.20;`	指定 Solidity 编译器版本
`contract HelloWorld { ... }`	定义一个合约
`string public message`	状态变量，`public` 自动生成 getter
`function setMessage(...)`	函数，修改状态
`memory`	数据位置（字符串、数组等引用类型必须指定）
`view`	表示函数只读取状态，不修改

三、核心语法详解

1. 数据类型

类型	说明
`bool`	`true` / `false`
`int` / `uint`	有符号/无符号整数（默认 256 位） e.g. `int256`, `uint8`
`address`	存储以太坊地址（20 字节） `address payable` 可转账
`string`	动态字符串（存储在 storage）
`bytes`	固定或动态字节数组（`bytes32`, `bytes memory`）
`enum`	枚举类型
`struct`	自定义结构体

```
enum Status { Pending, Shipped, Delivered }
struct User {
    string name;
    uint age;
}
```

---

2. 状态变量与数据位置

Solidity 中变量有三种**数据位置**：

位置	说明
`storage`	持久化存储在区块链上（合约变量默认）
`memory`	临时内存，函数调用期间存在（如参数、局部变量）
`calldata`	不可修改的外部调用数据（类似 memory，但更省 gas）

注意：`string`, `bytes`, `array`, `struct` 作为参数或局部变量时必须指定位置。

---

3. 函数（Functions）

```
function transfer(address to, uint amount) public payable {
    require(msg.value == amount, "Incorrect ETH sent");
    payable(to).transfer(amount);
}
```

函数修饰符（Modifiers）：

修饰符	说明
`public`	任何人都可调用
`private`	仅合约内部可调用
`internal`	仅本合约及子合约可调用
`external`	只能从外部调用
`view`	不修改状态（只读）
`pure`	不读也不写状态（纯函数）
`payable`	可接收 ETH

---

4. 事件（Events）

用于前端监听合约行为：

```
event Transfer(address indexed from, address indexed to, uint value);

function send(address to, uint amount) public {
    emit Transfer(msg.sender, to, amount);
}
```

- `indexed`：表示该参数可被过滤（最多 3 个）
- 前端可通过 Web3.js / Ethers.js 监听事件

---

5. 修饰器（Modifiers）

用于复用函数前置逻辑：

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}

function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

- `_;` 表示函数体执行位置

---

6. 继承（Inheritance）

```
contract Animal {
    function speak() public pure virtual returns (string memory);
}

contract Dog is Animal {
    function speak() public pure override returns (string memory) {
        return "Woof!";
    }
}
```

- `virtual`：允许子类重写
- `override`：重写父类函数

---

7. 错误处理

```
require(condition, "Error message"); // 条件不满足时 revert，返还剩余 gas
assert(condition); // 严重错误（如算法错误），消耗所有 gas
revert("Custom error"); // 手动回滚
```

自定义错误（Solidity 0.8+）

```
error InsufficientBalance(uint available, uint required);

function withdraw(uint amount) public {
    if (amount > balance) {
        revert InsufficientBalance({available: balance, required: amount});
    }
}
```

比字符串更省 gas。

四、常用模式与最佳实践

1. Ownable 模式（权限控制）

```
address public owner;

constructor() {
    owner = msg.sender;
}

modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}
```

2. 金额比较使用 SafeMath?

> ⚠ Solidity 0.8+ 已内置**溢出检查**，无需 `SafeMath` 库！

```
uint a = 1;
a--; // 会 revert（下溢）
```

3. 接收 ETH

```
// 接收 ETH（空函数）
receive() external payable {}

// 回退函数（fallback）
fallback() external payable {}
```

4. 发送 ETH

```
// 推荐：使用 call（更安全）
(bool success, ) = payable(to).call{value: amount}("");
require(success, "Transfer failed");
```

> ✕ 避免使用 `.send()` 和 `.transfer()`（gas 限制已被弃用）

五、实战：ERC-20 代币示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IERC20 {
    function totalSupply() external view returns (uint);
    function balanceOf(address account) external view returns (uint);
    function transfer(address to, uint amount) external returns (bool);
    function allowance(address owner, address spender) external view returns
(uint);
    function approve(address spender, uint amount) external returns (bool);
    function transferFrom(address from, address to, uint amount) external
returns (bool);

    event Transfer(address indexed from, address indexed to, uint value);
    event Approval(address indexed owner, address indexed spender, uint value);
}

contract SimpleToken is IERC20 {
    string public name = "Simple Token";
    string public symbol = "STK";
    uint8 public decimals = 18;
    uint  public override totalSupply;

    mapping(address => uint) public override balanceOf;
    mapping(address => mapping(address => uint)) public override allowance;

    constructor(uint initialSupply) {
        totalSupply = initialSupply * 10**decimals;
```

```
        balanceOf[msg.sender] = totalSupply;
    }

    function transfer(address to, uint amount) public override returns (bool) {
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
    }

    function approve(address spender, uint amount) public override returns
(bool) {
        allowance[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
        return true;
    }

    function transferFrom(address from, address to, uint amount) public override
returns (bool) {
        allowance[from][msg.sender] -= amount;
        balanceOf[from] -= amount;
        balanceOf[to] += amount;
        emit Transfer(from, to, amount);
        return true;
    }
}
```

六、安全注意事项

1. **重入攻击**：使用 **Checks-Effects-Interactions** 模式
2. **整数溢出**：0.8+ 已内置检查
3. **权限控制**：使用 `onlyOwner`、`require` 等
4. **避免使用** `tx.origin`：应使用 `msg.sender`
5. **测试充分**：使用 Hardhat 编写单元测试
6. **代码审计**：上线前请专业团队审计

参考资料

- 官方文档：https://docs.soliditylang.org
- Solidity by Example：https://solidity-by-example.org
- Ethers.js：https://docs.ethers.org
- OpenZeppelin Learn：https://www.openzeppelin.com/learn